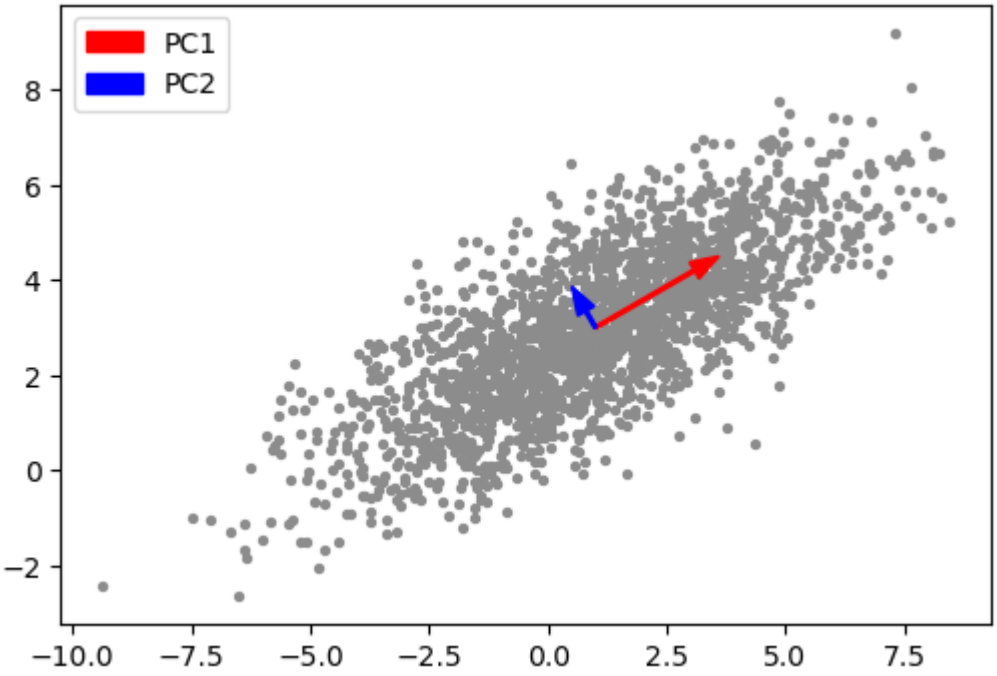
HW6 Coding Part

Problem 0: Set up (0 points)

The following code generates a set of sample points from a two-dimensional Gaussian distribution and displays them as a scatter plot. The distribution is centered at (1.0, 3.0), with its first 2 principal direction are shown in the figure. The standard deviation along this direction is 3, and 1 in the orthogonal direction, resulting in an elliptical spread of the samples that exhibits clear directional structure and scale difference. Here we use PC as the abbreviation for principal component.

In [7]:
```
from utils import generate_data
samples = generate_data()
```



Problem 1: PCA (5 points)

Now we aim to manually implement PCA using the `samples` data generated in `Problem 0`, and demonstrate the so-called "dimensionality reduction" process.

Although in this case the data remains two-dimensional (i.e., 2D to 2D) and no actual reduction in dimensionality occurs, we still refer to it as "dimensionality reduction" to stay consistent with the terminology used in later sections where true reduction will be performed.

In this problem, your are required to implement PCA by yourself, you can use the `numpy` library to help you with the computation, but you are not allowed to use any built-in functions or libraries that directly perform PCA.

In [8]:
```python
import numpy as np
def PCA(data: np.ndarray, n_components: int = 2):
    # 1. Center the data
    # TODO: Your code here
    mean = np.mean(data, axis=0)
    centered_data = data - mean

    # 2. Use SVD to compute the eigenvalues and eigenvectors
    # TODO: Your code here
    U, s, Vh = np.linalg.svd(centered_data, full_matrices=False)

    # 3. Collect the top 'n_components' eigenvectors, and then project th
    # TODO: Your code here
    top_components = Vh[:n_components, :] # Select the first n_components
    projected_data = centered_data @ top_components.T

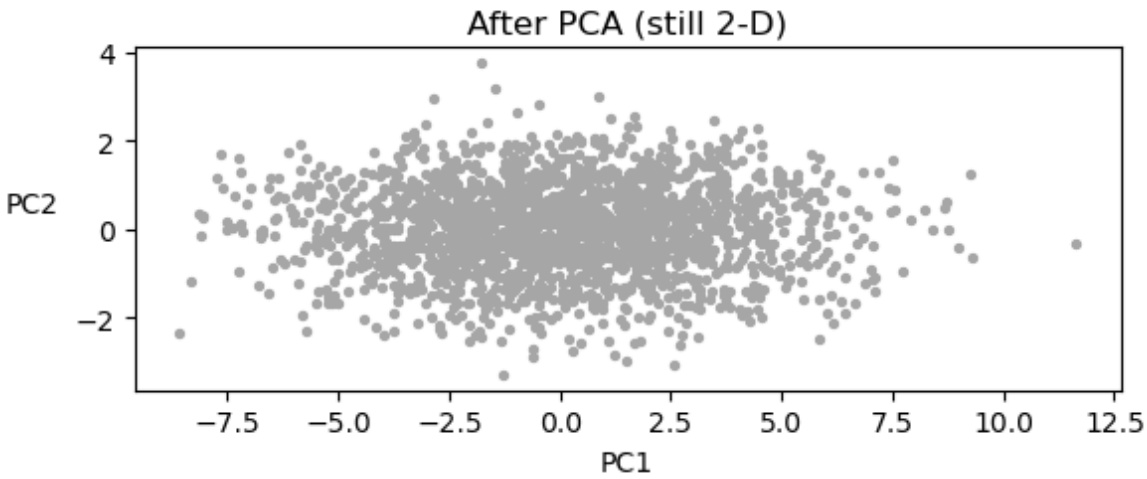
    return projected_data
```

After implement the PCA algorithm, we can visualize whether the data is successfully reduced to 2D and then project the data onto the principal axes.

In [9]:
```python
import matplotlib.pyplot as plt

projected_data = PCA(samples, n_components=2)

plt.scatter(projected_data[:, 0], projected_data[:, 1], s=8, color='0.65'
plt.title("After PCA (still 2-D)")
plt.xlabel("PC1"); plt.ylabel("PC2", rotation=0, labelpad=15)
plt.gca().set_aspect("equal")
plt.show()
```



Problem 2: Visualize Latent Space (15 points)

PCA is a commonly used technique for reducing high-dimensional data into two or three dimensions for visualization. In the context of neural networks, it can help us understand what high-level features (feature maps) have been learned by projecting them onto a space we can see and interpret.

In this example, we use the LeNet architecture mentioned in Recitation 8 (you can check the network details in utils, or refer to the files on piazza resources about

recitation 8). After passing through several convolution and pooling layers, each input image is transformed by the fully connected layer fc1 into a high-dimensional representation known as the latent space. This is followed by another fully connected layer fc2 that maps the latent space features to a 10-dimensional output for classification.

A well-trained classifier typically extracts highly informative features in the latent space — that is, the latent features should reflect the class of the input image well. However, since the latent space is often high-dimensional (e.g., 64 dim in ours example), we can't visualize it directly. To address this, we use PCA to reduce the features to 2D.

If your PCA implementation is correct and the classifier is effective, you should observe that the projected features form clear clusters in 2D space, where samples with the same label are close together and samples with different labels are well separated. We have already trained a well-behavior classifier for you, you can just load the model 'mnist_cnn.pth' to get the models parameters.

Now, please follow the instructions and complete the code to implement this visualization.

In [10]:
```
import torch.utils.data.dataloader
from utils import CNN
import torch
from torchvision import datasets, transforms
import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn

# ---------- load checkpoint ----------
# TODO: Load the checkpoint of the CNN model, please make sure you are us
# Your code here
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CNN()  # Instantiate the model from utils.py
model.load_state_dict(torch.load('mnist_cnn.pth', map_location=device))
model = model.to(device)
model.eval()  # Set model to evaluation mode

# ---------- prepare dataset ----------
dataloader = torch.utils.data.DataLoader(datasets.MNIST(root="./data", tr
print("load data successfully")

# ---------- get the features in latent space(features after the layer fc
features, labels = [], []

# Define a hook to extract features from the fc1 layer
activation = {}
def get_activation(name):
    def hook(model, input, output):
        activation[name] = output.detach()
    return hook

# Register the hook to get the output of fc1 layer
model.fc1.register_forward_hook(get_activation('fc1'))
```

```
with torch.no_grad():
    for x, y in dataloader:
        # TODO: Get the features of the latent space(after the layer fc1)
        # Your code here
        x = x.to(device)
        # Forward pass through the model
        _ = model(x)
        # Get the features from the hook
        feature = activation['fc1']
        features.append(feature.cpu().numpy())
        labels.append(y.numpy())

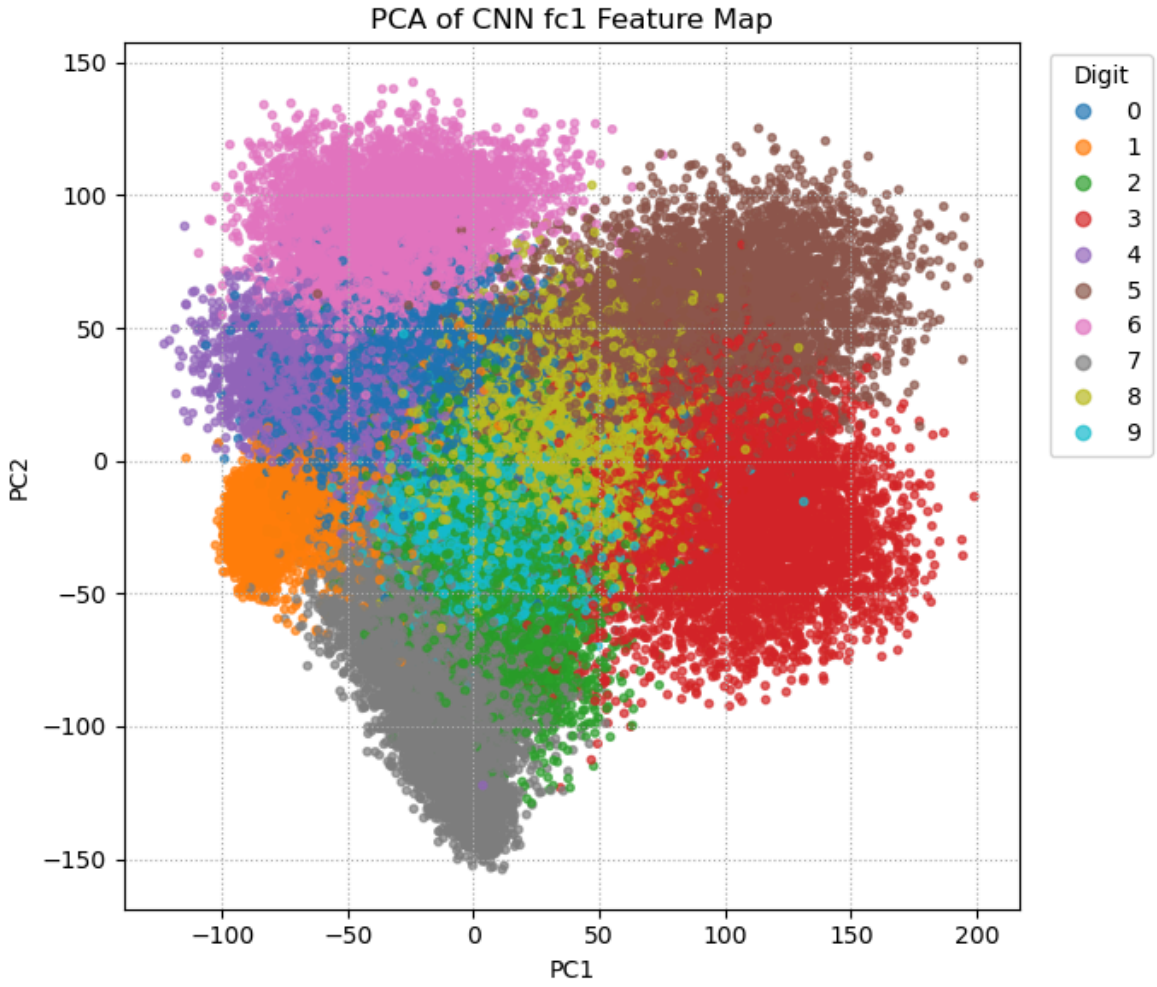
# TODO: concatenate all gathered features and labels, and name them as X
# Your code here
X = np.concatenate(features, axis=0)
y = np.concatenate(labels, axis=0)

# TODO: Reduce the dimension of the features to 2, and name the reduced f
# Hints: You can use the PCA function you have implemented in Problem 1
# Your code here

Z = PCA(X, n_components=2)
Z = -Z

# ---------- show the latent space in 2D ----------
plt.figure(figsize=(7, 6))
scatter = plt.scatter(Z[:, 0], Z[:, 1], c=y, cmap='tab10', s=10, alpha=0.
plt.legend(*scatter.legend_elements(), title="Digit", bbox_to_anchor=(1.0
plt.title("PCA of CNN fc1 Feature Map")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.grid(True, linestyle=":")
plt.tight_layout()
plt.show()
```

```
load data successfully
```



Problem 3: K-Means (10 points)

In Problem 2, we used PCA to reduce the high-dimensional latent features extracted by a neural network (such as LeNet) to a two-dimensional space, and successfully visualized how the samples are distributed in that space. We observed that samples belonging to the same class tend to cluster closely in the principal component space, suggesting that the extracted features are quite separable.

Next, we further explore whether it's possible to distinguish between different classes using only these low-dimensional representations. To do this, we will apply K-Means clustering on the 2D latent features as an unsupervised learning method. K-Means will attempt to partition all the samples into k=10 clusters, each potentially corresponding to an underlying class.

Now, please try implementing it yourself.

In [11]:
```python
def kmeans(X, k=10, max_iter=200, tol=1e-4, seed=0):
    rng = np.random.default_rng(seed)
    N = X.shape[0]

    # 1. Initialize: Choose k points as initial centers, you can use any
    centers = rng.choice(N, k, replace=False)
    centers = X[centers]

    for t in range(max_iter):
```

```
        # 2. Assign each point to the closest center
        # Calculate distances between each point and each center
        distances = np.zeros((N, k))
        for i in range(k):
            # Compute squared Euclidean distance for each point to center
            distances[:, i] = np.sum((X - centers[i])**2, axis=1)

        # Assign each point to the closest center
        labels = np.argmin(distances, axis=1)

        # 3. visualize the clustering result
        if t < 5 or t % 10 == 0:
            show_cluster(X, centers, labels, t)

        # 4. get the new centers
        new_centers = np.zeros_like(centers)
        for i in range(k):
            # Select all points assigned to cluster i
            cluster_points = X[labels == i]
            if len(cluster_points) > 0:
                # Update center as the mean of all points in the cluster
                new_centers[i] = np.mean(cluster_points, axis=0)
            else:
                # If no points assigned to this cluster, keep the old cen
                new_centers[i] = centers[i]

        # 5. calculate the shift distance of the centers
        # Compute the Euclidean distance between old and new centers
        shift = np.sqrt(np.sum((centers - new_centers)**2))

        # 6. Check whether converged
        if shift < tol:
            show_cluster(X, centers, labels, t)
            print(f"Converged at iteration {t}")
            break

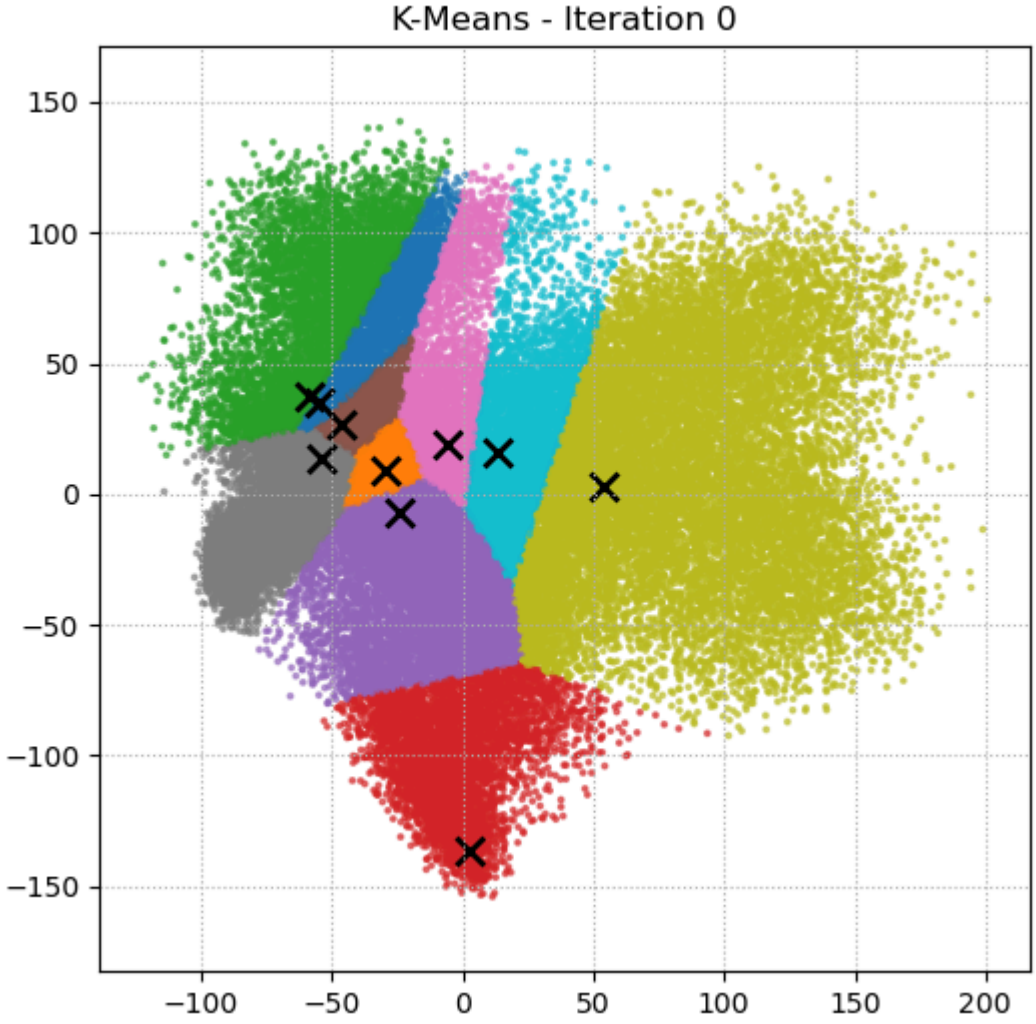
        centers = new_centers

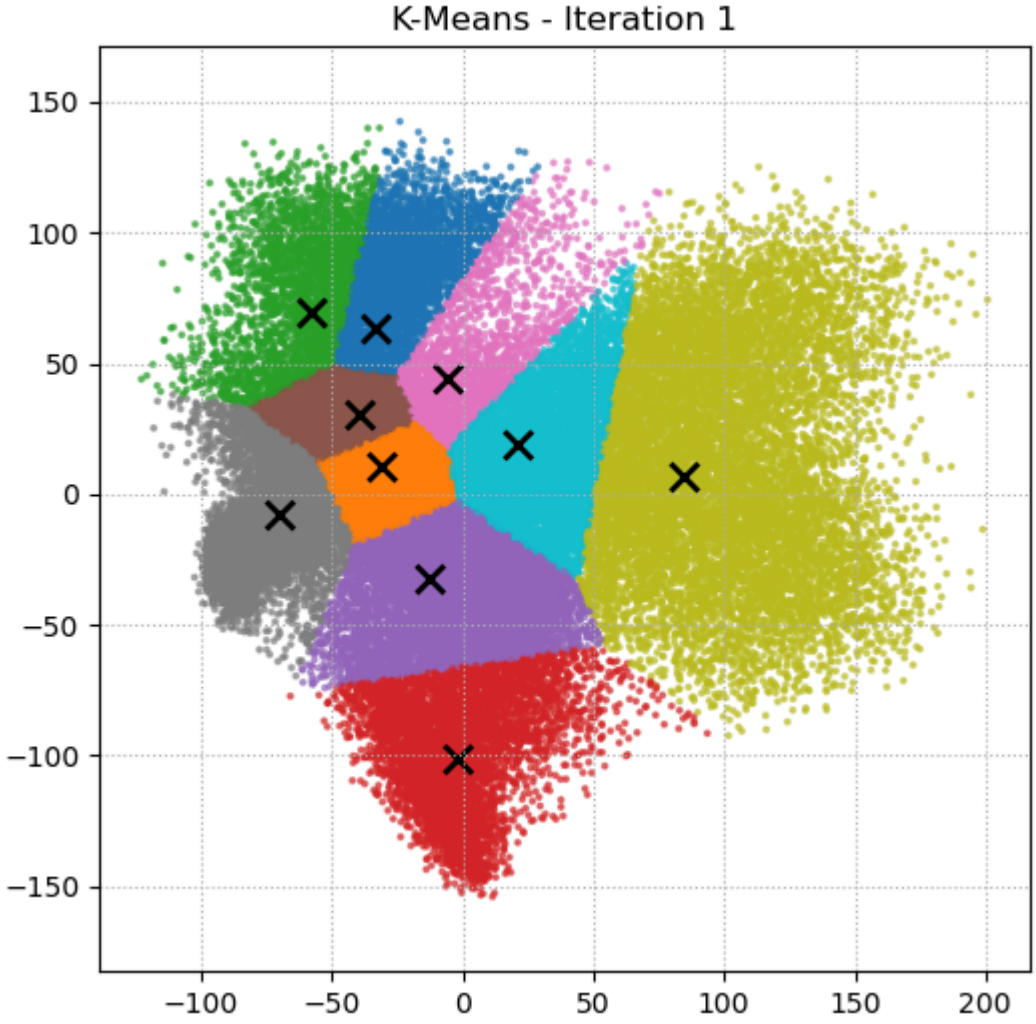
    return centers, labels
```

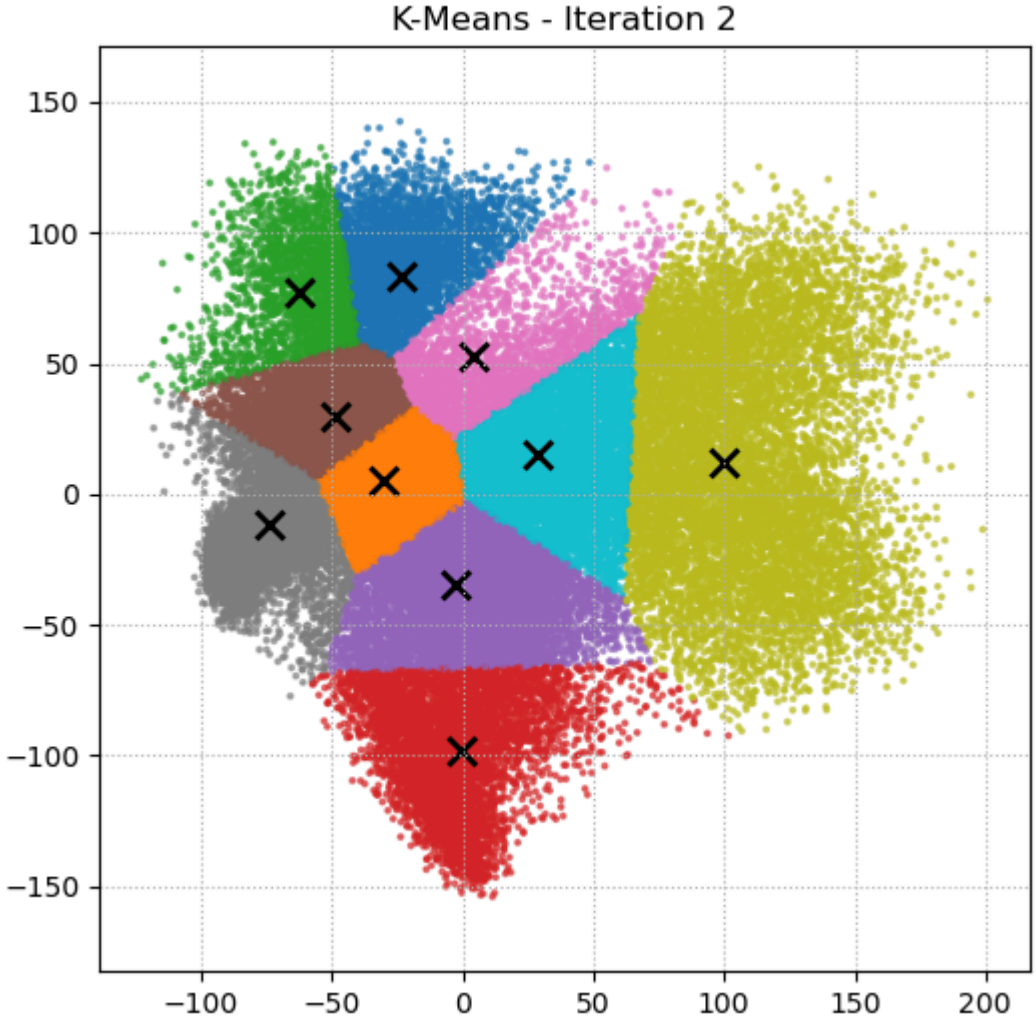
Run the K-Means algorithm on the reduced-dim latent space.

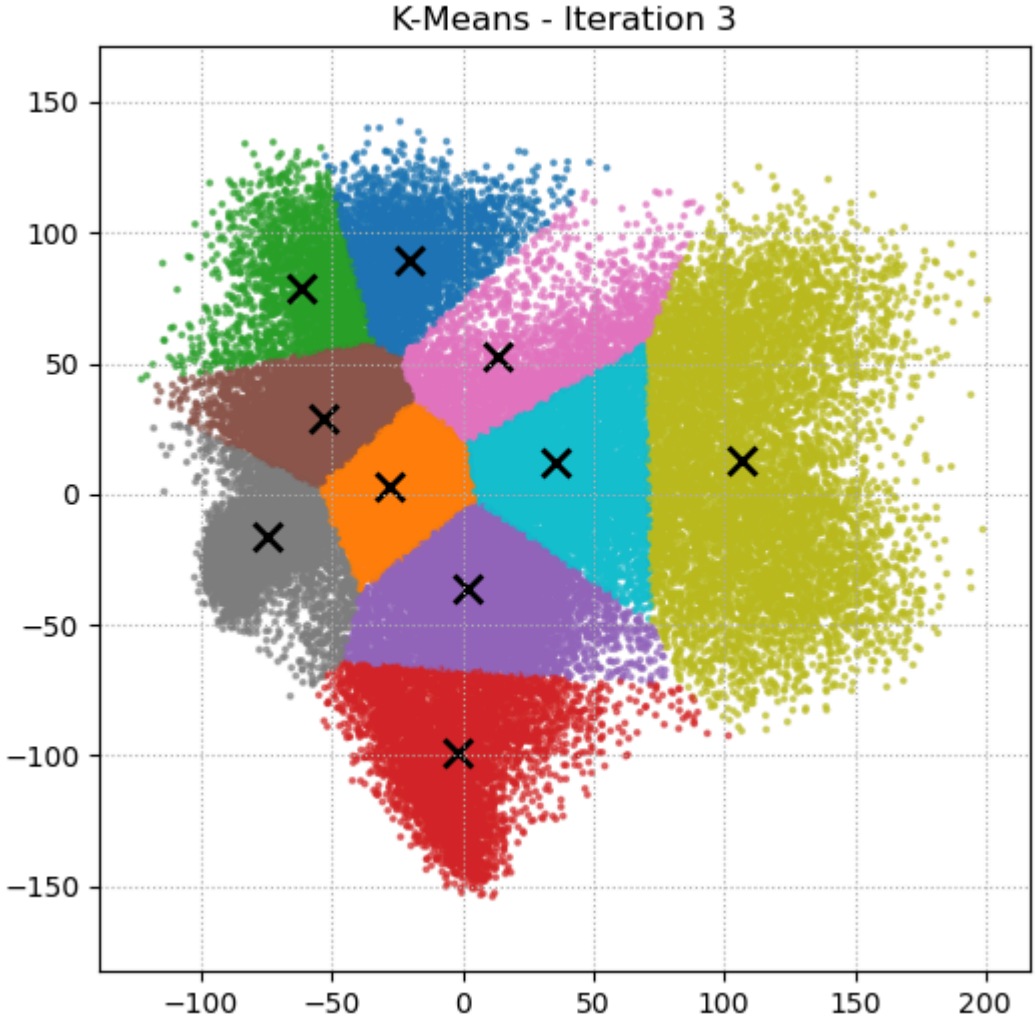
In [12]:

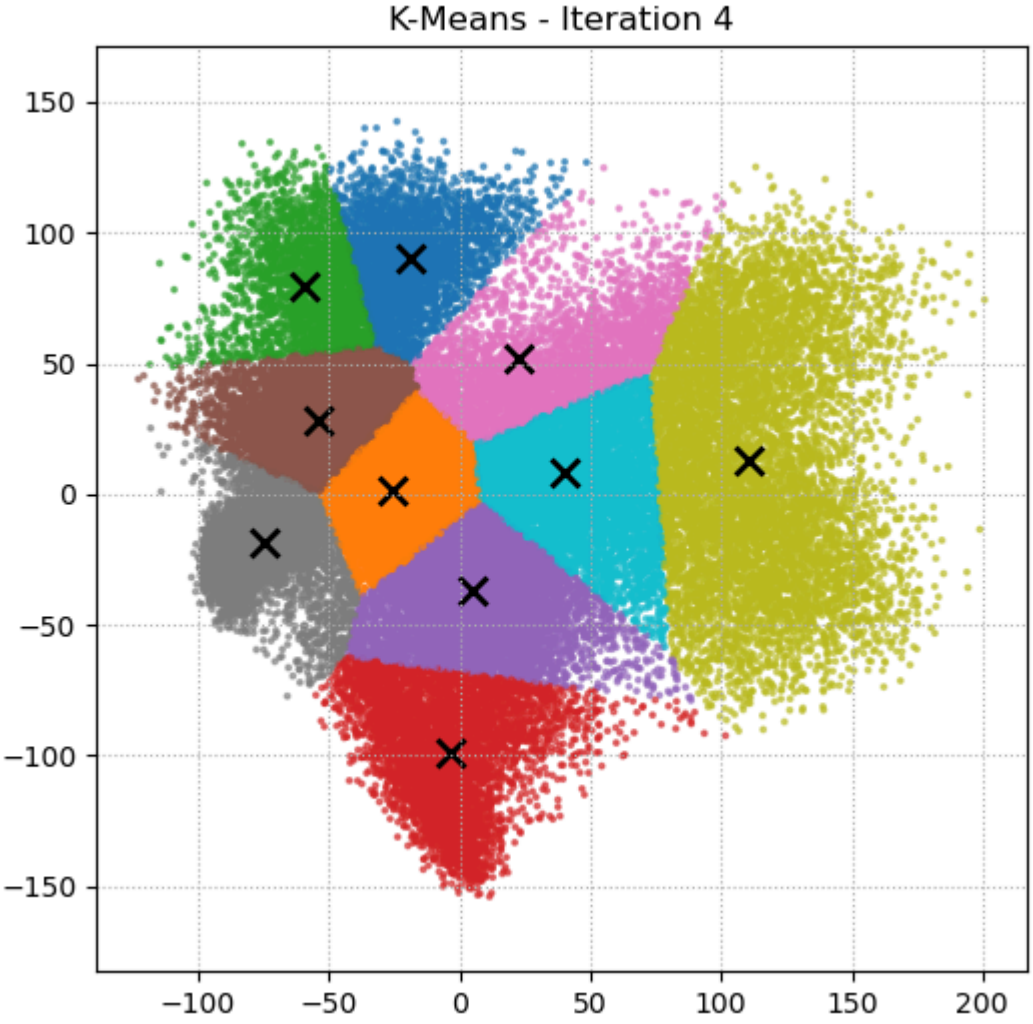
```
centers, _ = kmeans(Z)
print(centers)
```

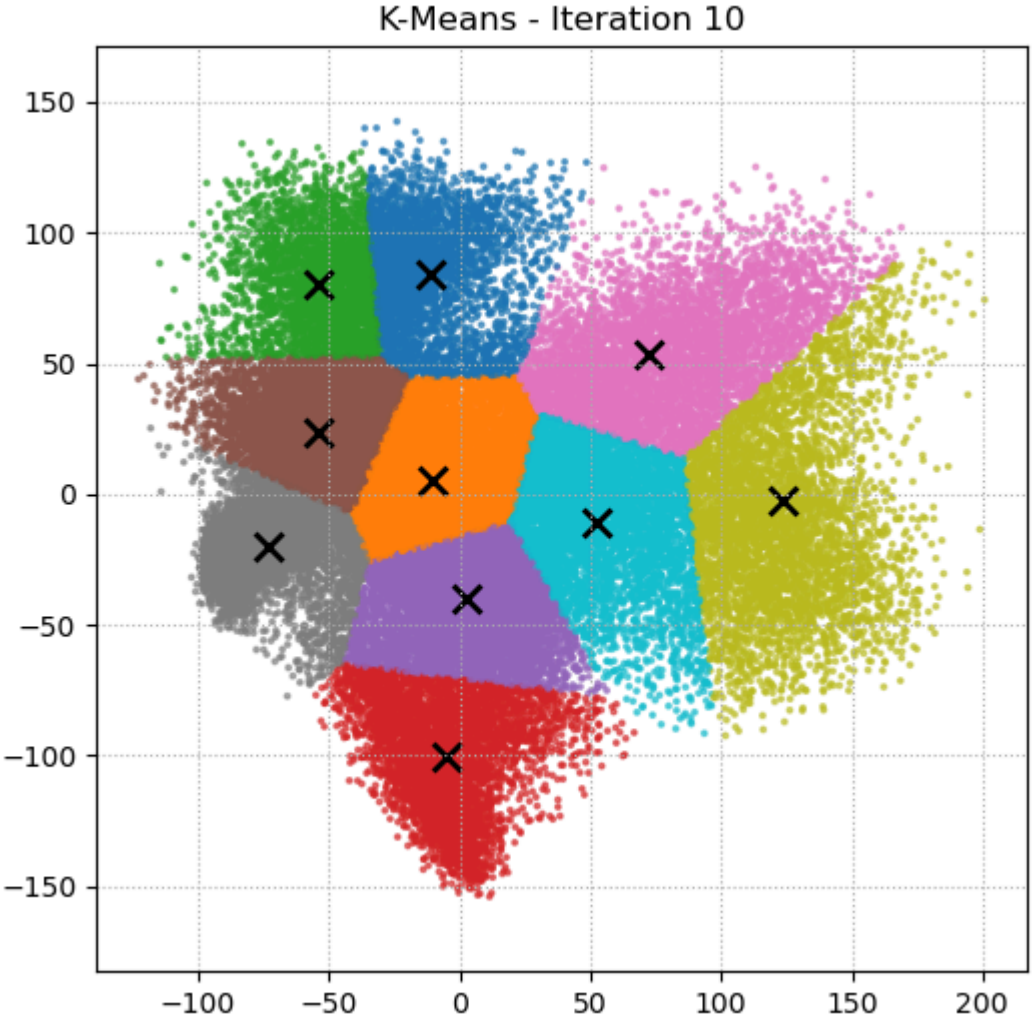

K-Means - Iteration 0
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

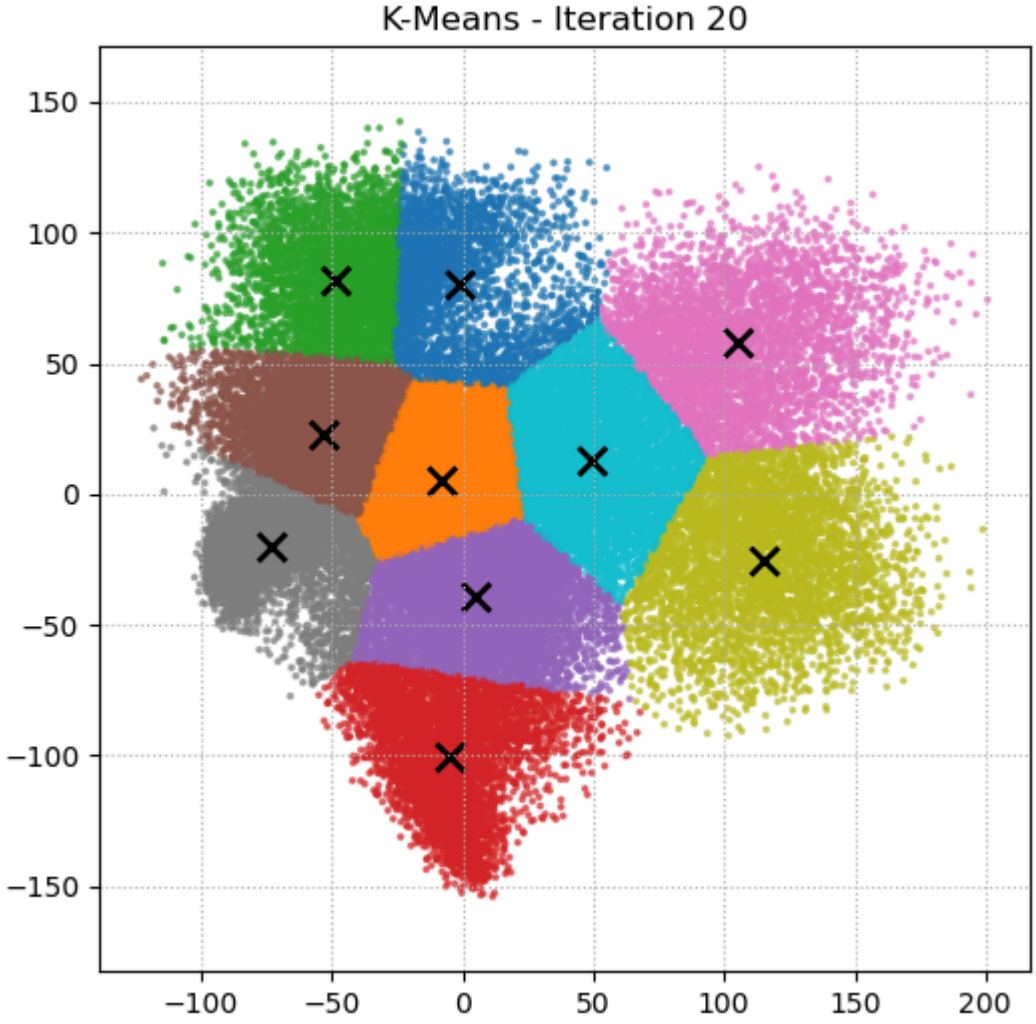

K-Means - Iteration 1
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

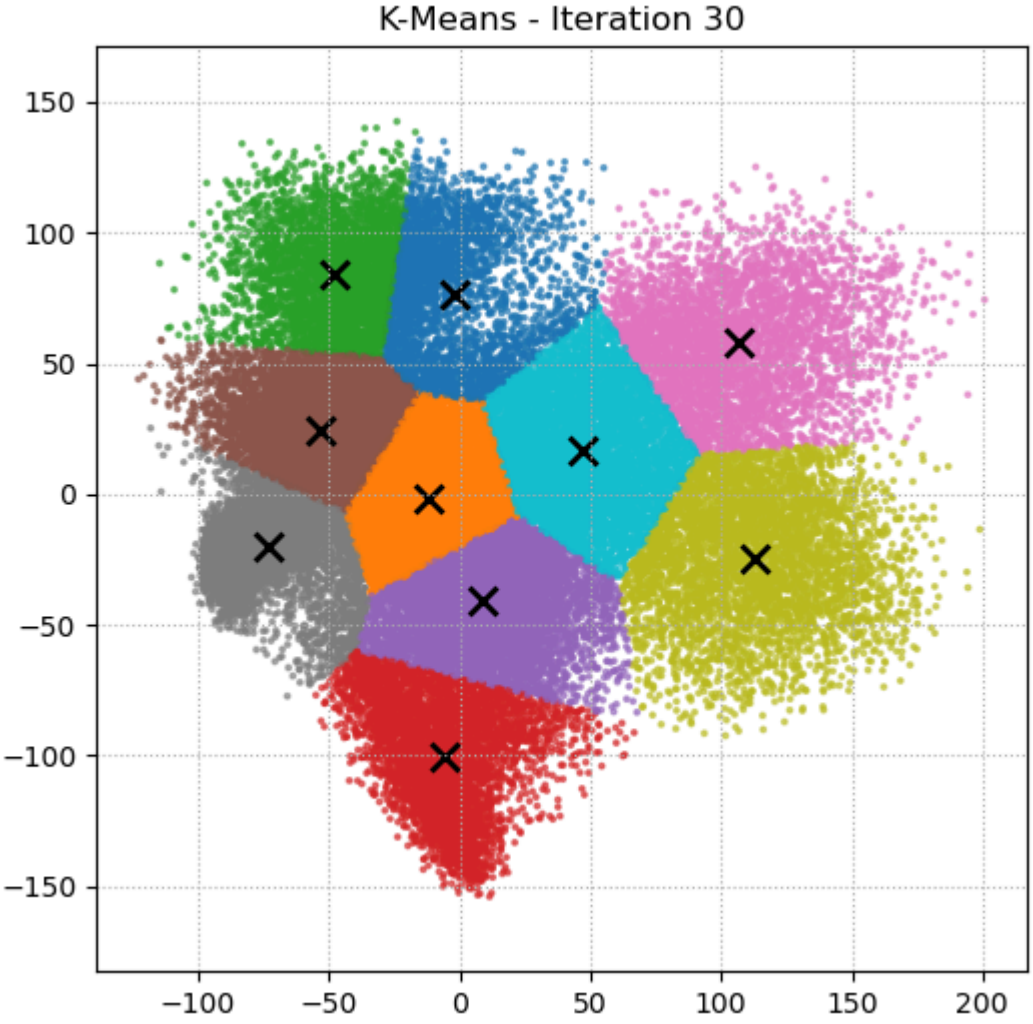

K-Means - Iteration 2
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

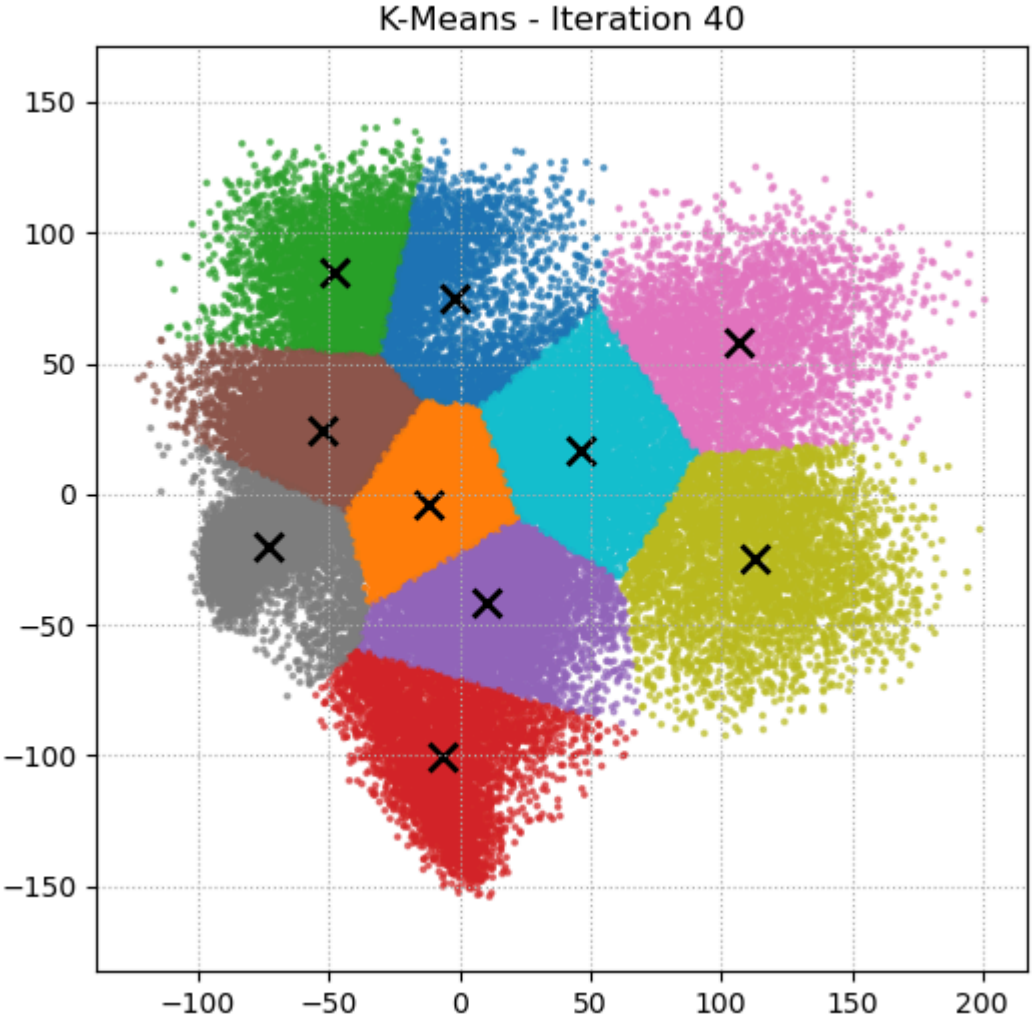

K-Means - Iteration 3
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

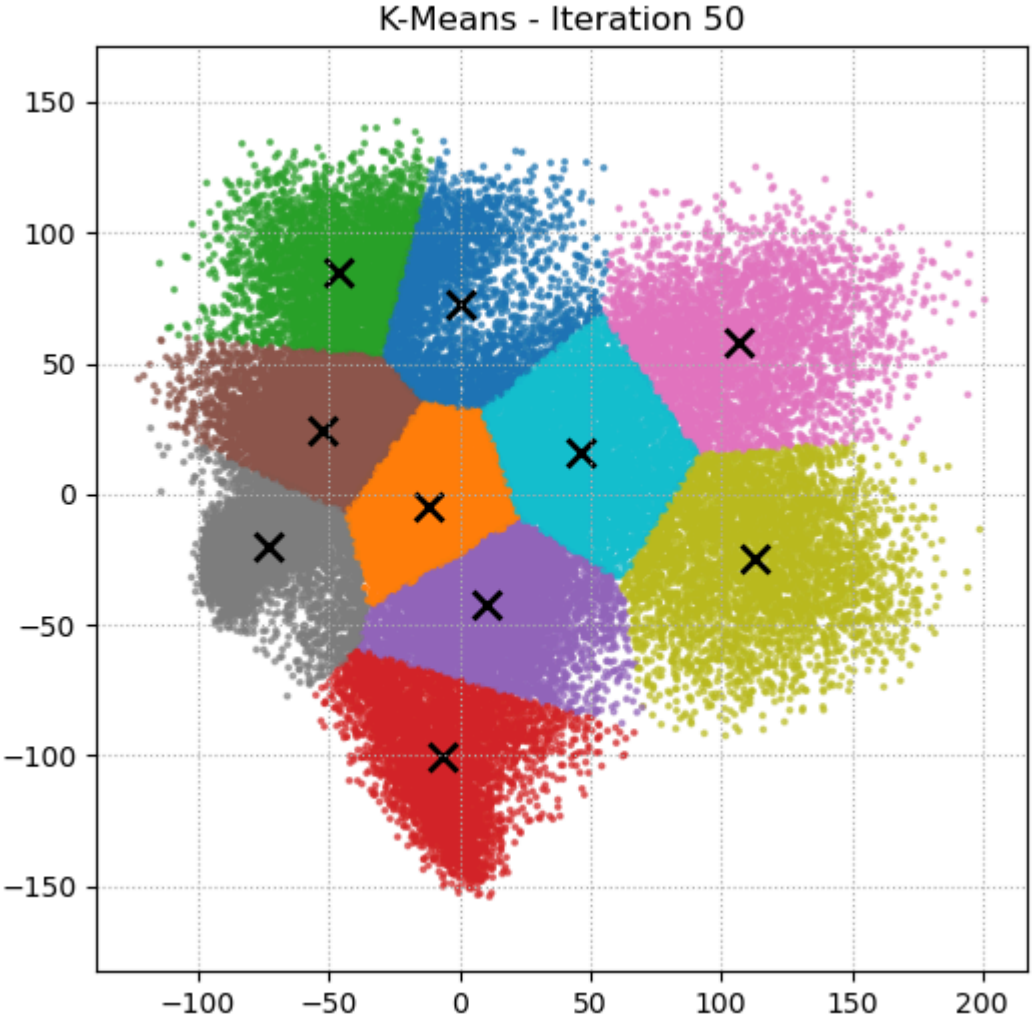

K-Means - Iteration 4
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

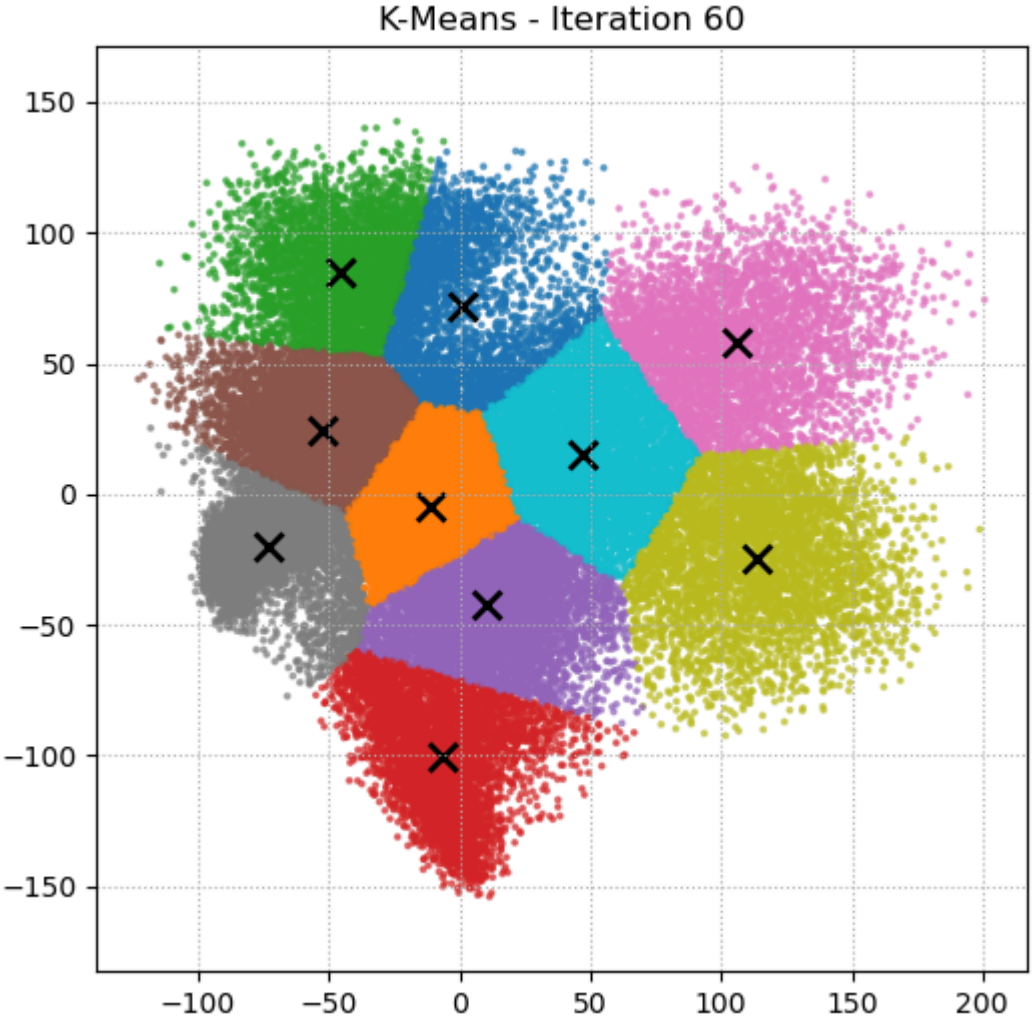

K-Means - Iteration 10
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

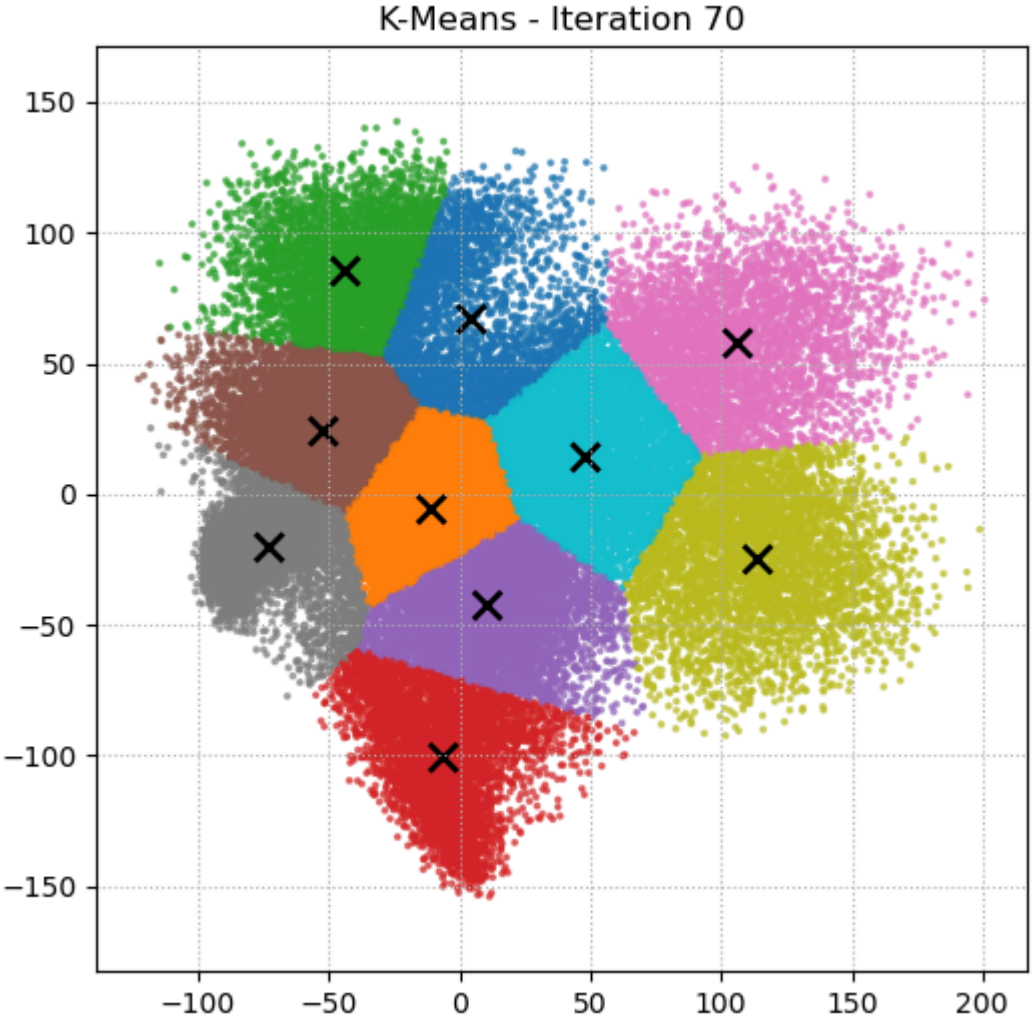

K-Means - Iteration 20
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

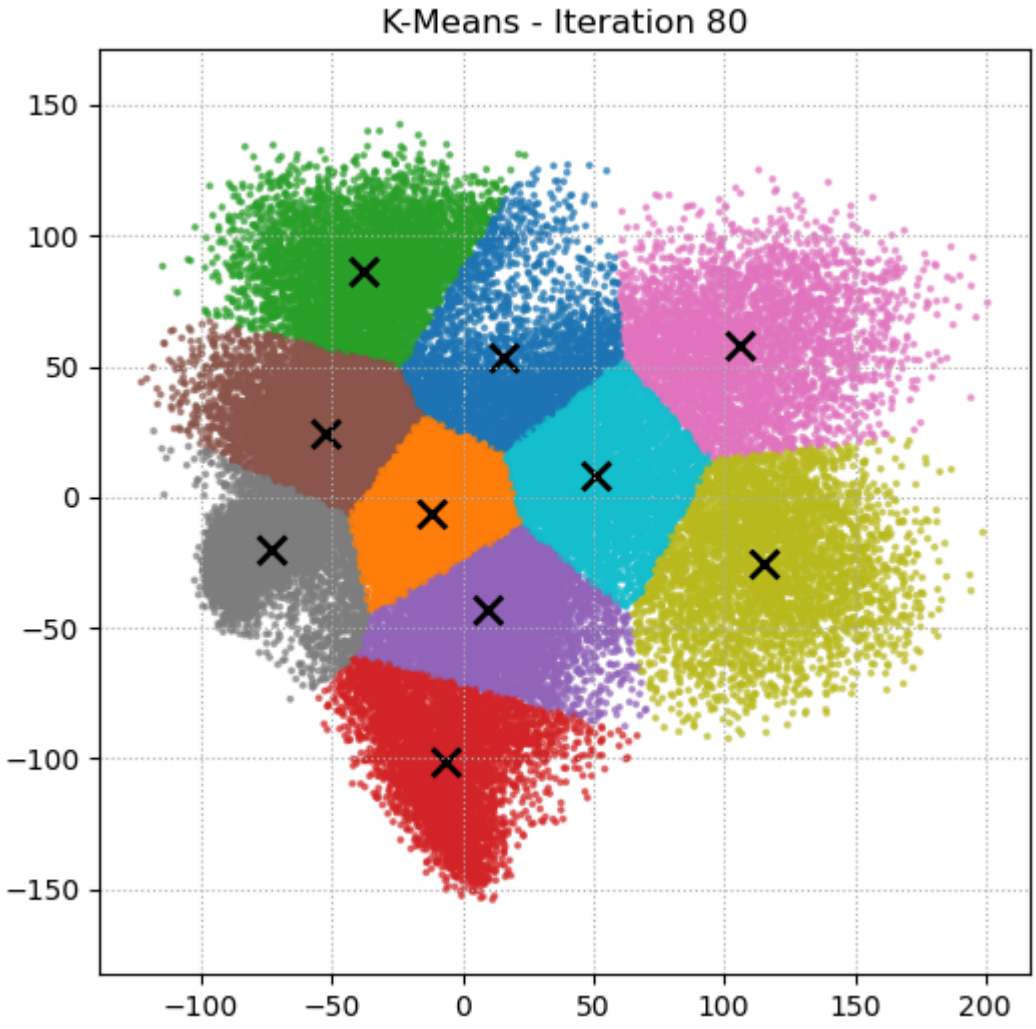

K-Means - Iteration 30
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

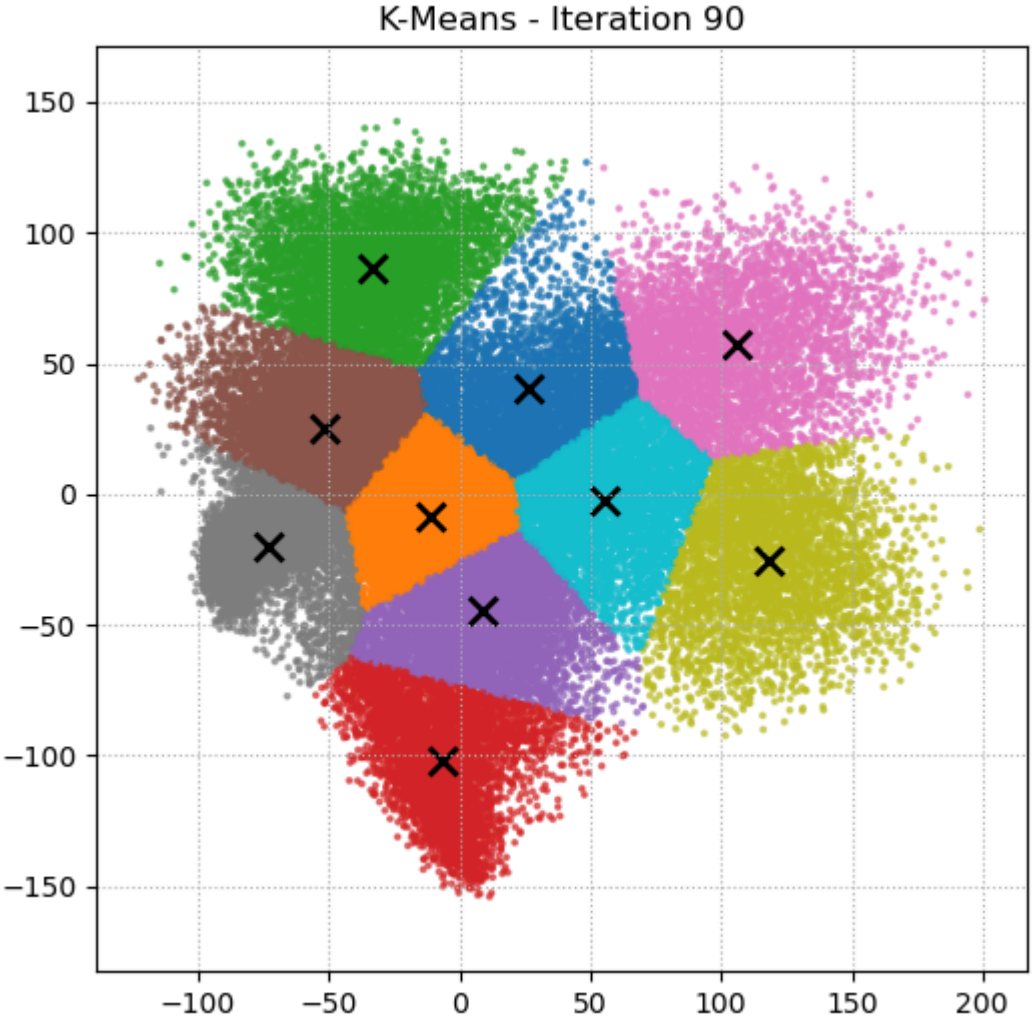

K-Means - Iteration 40
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

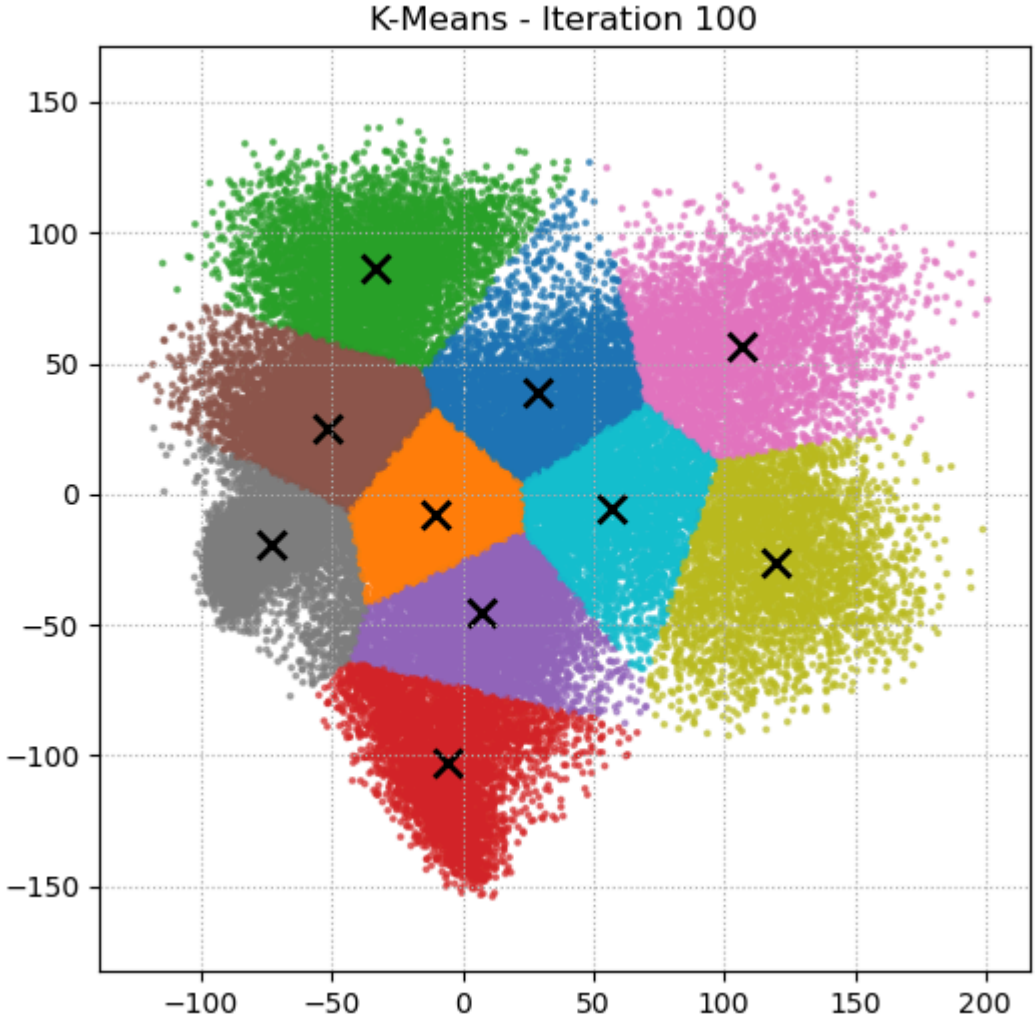

K-Means - Iteration 50
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

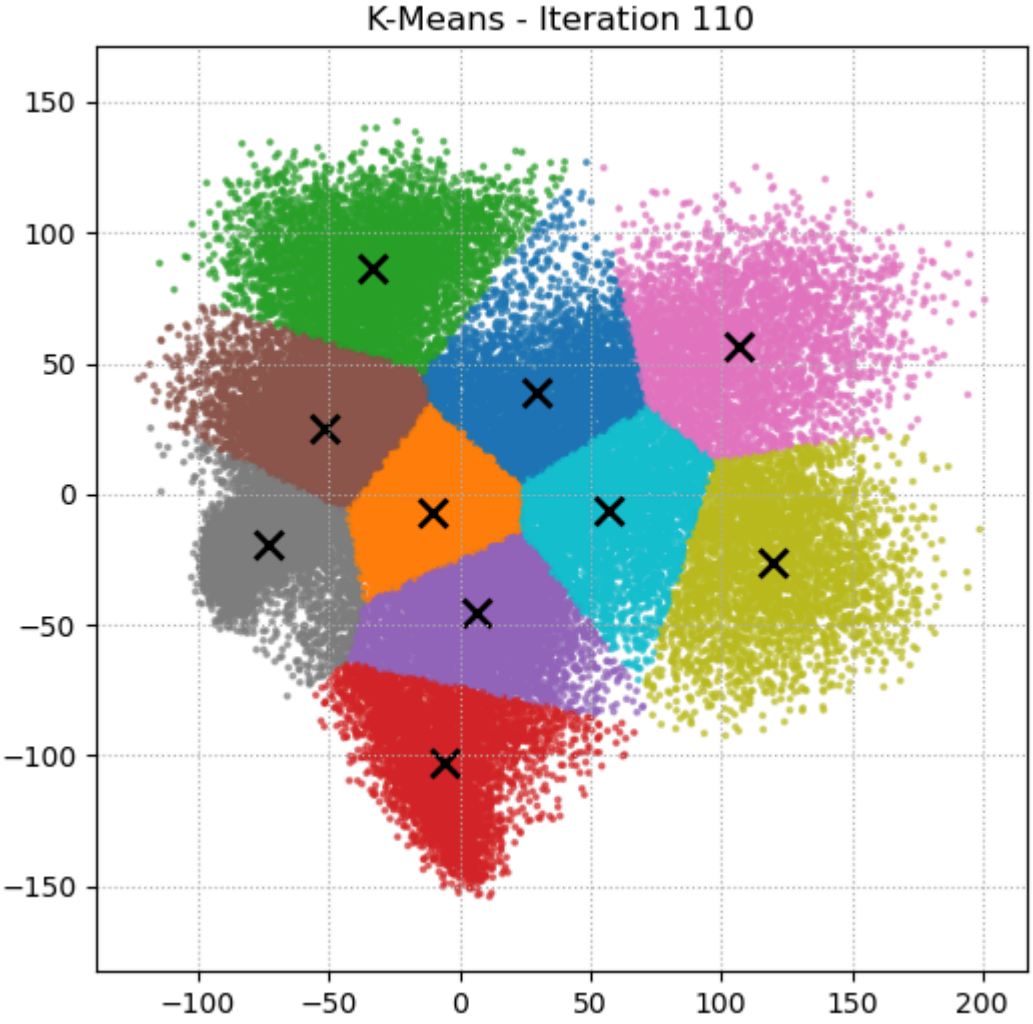

K-Means - Iteration 60
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

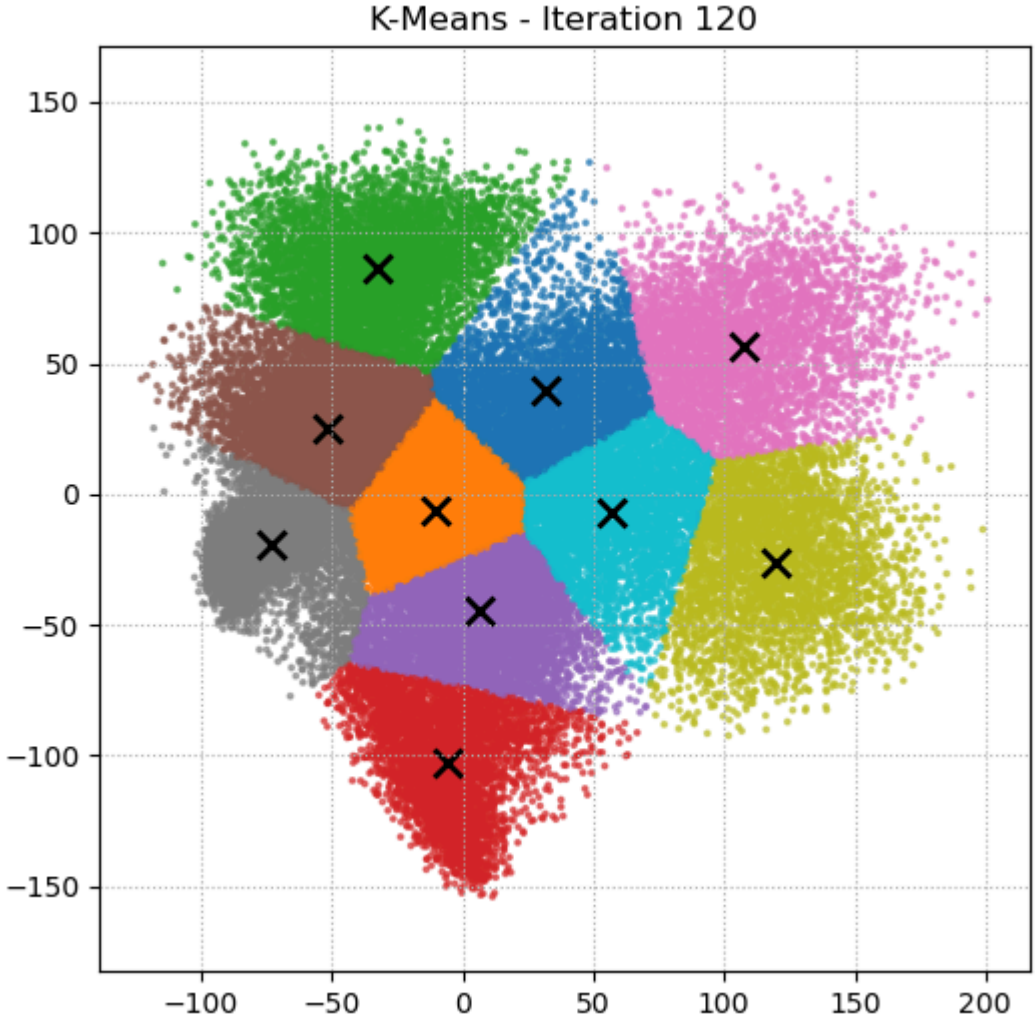

K-Means - Iteration 70
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

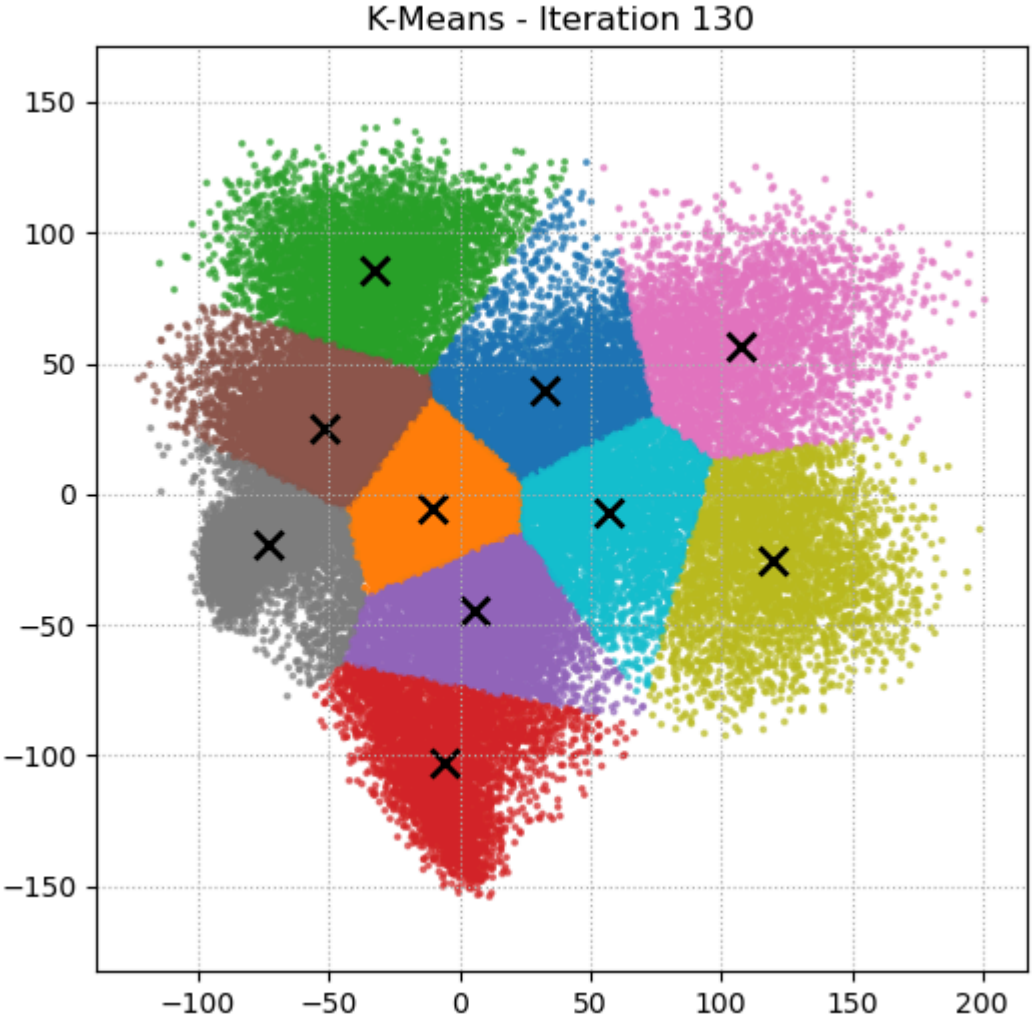

K-Means - Iteration 80
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

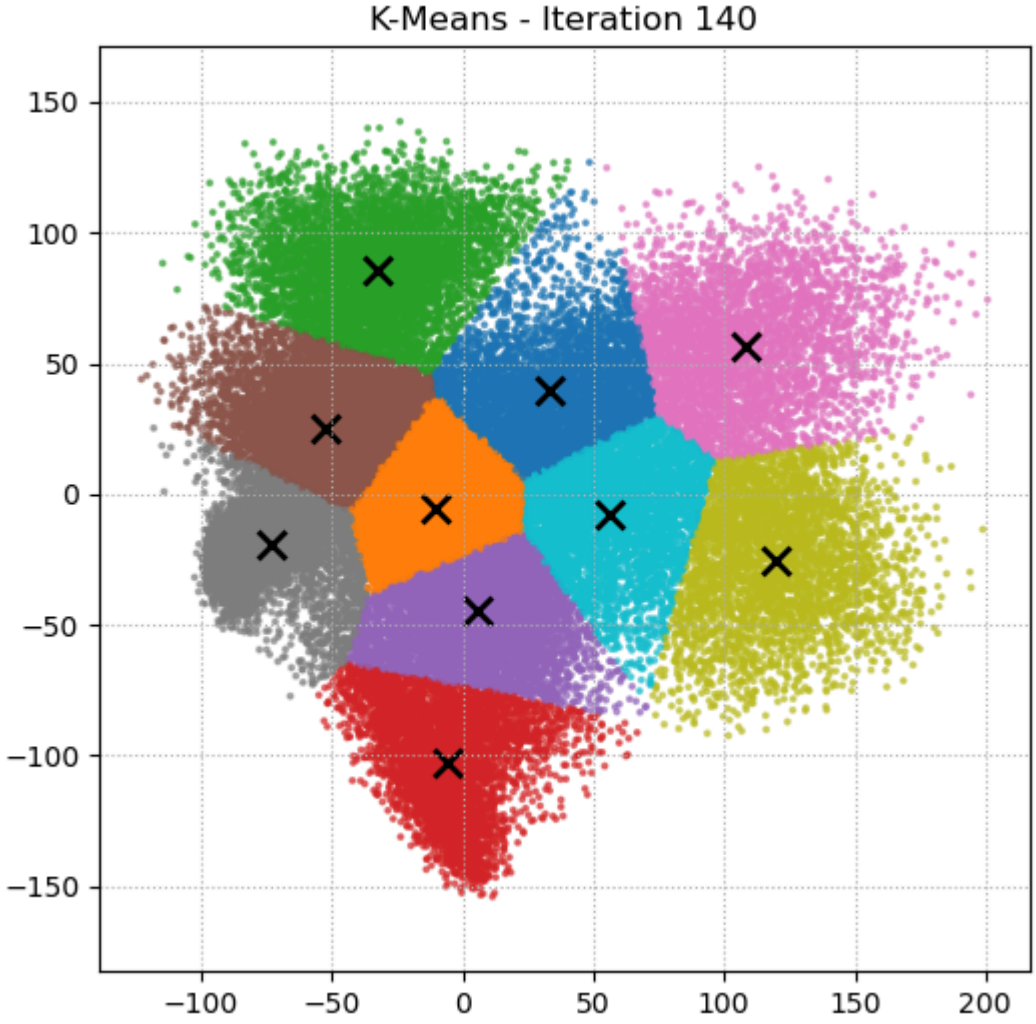

K-Means - Iteration 90
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

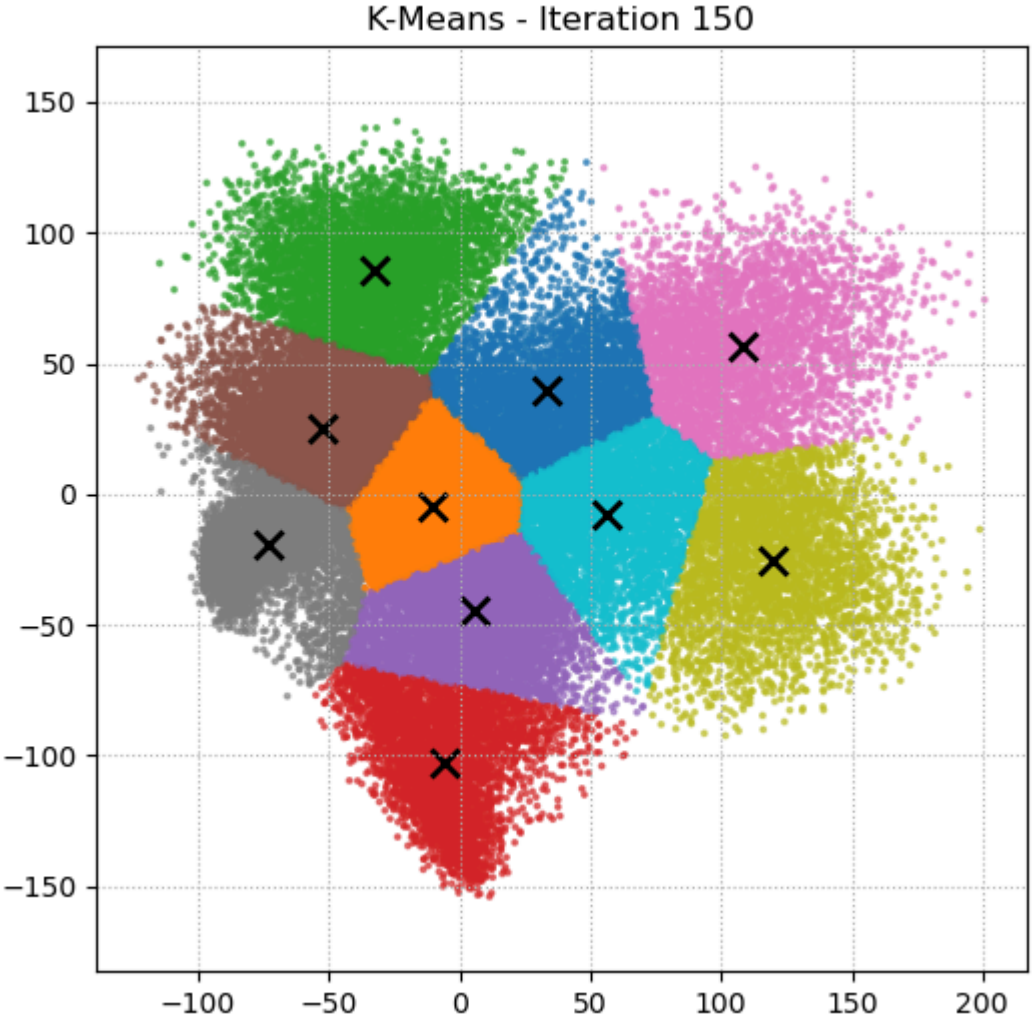

K-Means - Iteration 100
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

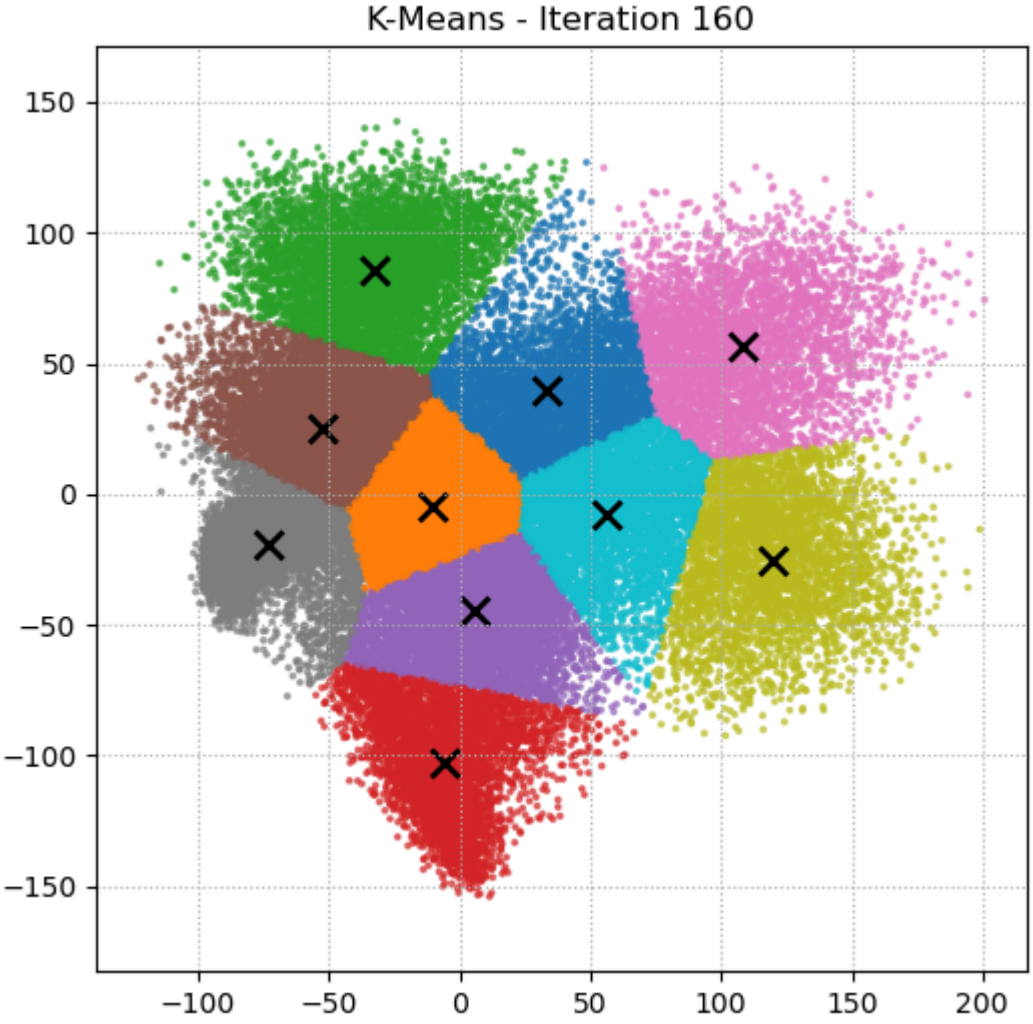

K-Means - Iteration 110
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200

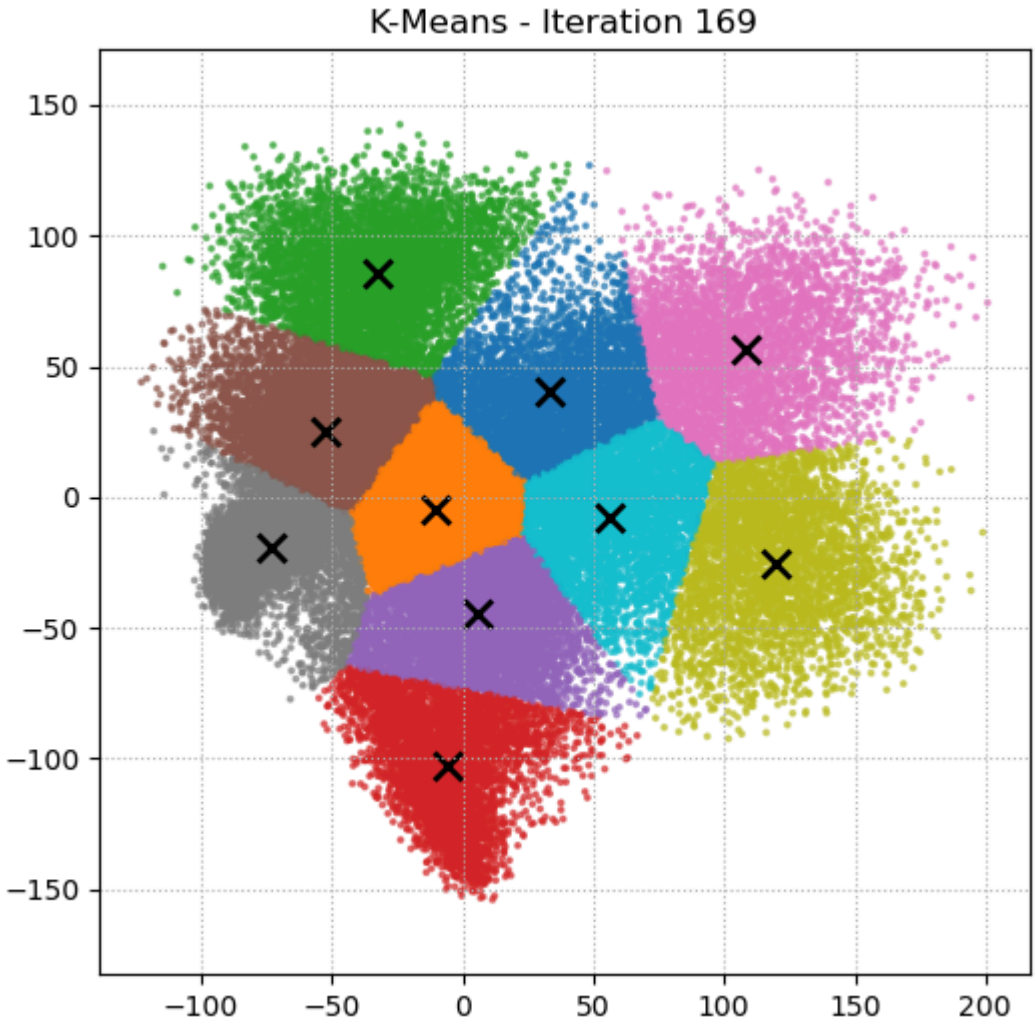

K-Means - Iteration 120
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200


K-Means - Iteration 130
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200


K-Means - Iteration 140
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200


K-Means - Iteration 150
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200


K-Means - Iteration 160
150
100
50
0
−50
−100
−150
−100
−50
0
50
100
150
200




```
Converged at iteration 169
[[  33.59375     40.10119  ]
 [ -10.31425     -4.8757453]
 [ -32.931976    85.98567  ]
 [  -5.7787185 -102.41468  ]
 [   5.4824185  -44.376266 ]
 [ -52.23849     25.429956 ]
 [ 108.285484    56.7125   ]
 [ -73.336205   -19.452301 ]
 [ 120.03384    -25.759636 ]
 [  56.45711     -7.504172 ]]
```